ASSIGNMENT 5

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: DSA0109

COURSE NAME: OBJECT ORIENTED PROGRAMMING WITH C++

PART A – DESIGN

TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY

A1.1 Identify the common interface

All hospital bills should have common information and operations.

Common data:

- Patient name
- Bill number
- Patient ID

Common operations:

- total() – calculates the total bill amount.
- type() – returns the type of bill.
- print() – displays bill details.

A1.2 Design the hierarchy

Create an abstract base class Bill with pure virtual functions. The derived classes are:

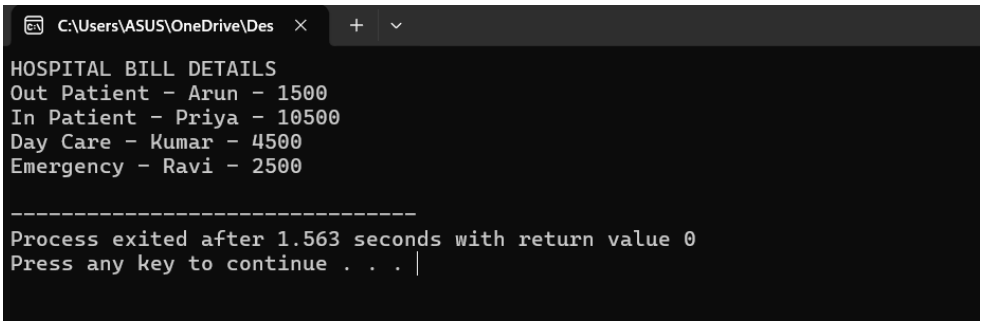
1. OutPatientBill
2. InPatientBill
3. DayCareBill
4. EmergencyBill

C++ class design:

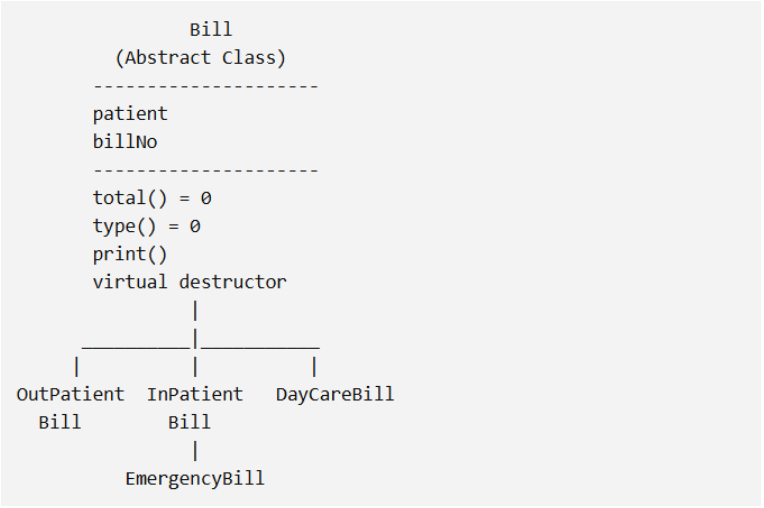
```cpp
#include <iostream>
#include <string>
using namespace std;

class Bill
{
protected:
    string patient;
    int billNo;
public:
    Bill(string p, int b) : patient(p), billNo(b) {}
    virtual double total() const = 0;
    virtual string type() const = 0;
    virtual void print() const
    {
        cout << type() << " - " << patient
             << " - " << total() << endl;
    }
    virtual ~Bill() = default;
};
class OutPatientBill : public Bill
{
    double consultation, tests;
public:
    OutPatientBill(string p, int b, double c, double t)
        : Bill(p, b), consultation(c), tests(t) {}
    double total() const override
    {
        return consultation + tests;
    }
    string type() const override
    {
        return "Out Patient";
    }
};
class InPatientBill : public Bill
{
    int days;
    double room, procedures, medicines;
public:
    InPatientBill(string p, int b, int d,
                  double r, double pr, double m)
        : Bill(p, b), days(d), room(r),
          procedures(pr), medicines(m) {}
    double total() const override
    {
        return days * room + procedures + medicines;
    }
    string type() const override
    {
        return "In Patient";
    }
};
class DayCareBill : public Bill
{
    double packagePrice;
public:
    DayCareBill(string p, int b, double price)
        : Bill(p, b), packagePrice(price) {}
    double total() const override
    {
        return packagePrice;
    }
    string type() const override
    {
        return "Day Care";
    }
};
class EmergencyBill : public Bill
{
    double baseCharge, hourlyCharge;
    int hours;
public:
    EmergencyBill(string p, int b, double base,
                  double hourly, int h)
        : Bill(p, b), baseCharge(base),
          hourlyCharge(hourly), hours(h) {}
    double total() const override
    {
        return baseCharge + hourlyCharge * hours;
    }
    string type() const override
    {
        return "Emergency";
    }
};
```

Output



UML class diagram:



The four bill classes inherit from the abstract Bill class. total() and type() are pure virtual functions, and each derived class overrides them. print() and the destructor are virtual functions.

A1.3 Process all bills together

Pseudocode:

```
ALGORITHM CalculateDailyTotal(bills, n)

BEGIN
    SET dailyTotal = 0

    FOR i = 0 TO n - 1
        dailyTotal = dailyTotal + bills[i]->total()
    END FOR

    DISPLAY dailyTotal

END
```

Alternative concept – One Bill structure with switch

A single Bill structure can store a type code and all possible bill details.

```
double total()
{
    switch (type)
    {
        case 1:
            return consultation + tests;

        case 2:
            return days * roomCharge + procedures + medicines;

        case 3:
            return packagePrice;

        case 4:
            return baseCharge + hourlyCharge * hours;

        default:
            return 0;
    }
}
```

Complexity comparison:

Feature	Inheritance	Single class with switch
Processing n bills	O(n)	O(n)
Time per bill	O(1) virtual call	O(1) switch
Space per bill	Common and derived data	May contain unused fields
Adding a new bill	Add a derived class	Modify switch and class
Maintenance	Easier	Becomes difficult as types increase

TASK A2 – UNDERSTAND RUNTIME BINDING

A2.1 Early binding and late binding

Early binding:

- Function call is decided during compilation.
- Used with normal non-virtual functions.
- Depends on the declared type of the object or pointer.

Late binding:

- Function call is decided during program execution.
- Used with virtual functions.
- Depends on the actual object type.

Example:

```
Bill* b = new OutPatientBill("Ravi", 101, 500, 1200);
cout << b->total();
```

Since total() is virtual, the OutPatientBill version is executed.

A2.2 Virtual table

A vtable is a compiler-generated table used to support runtime polymorphism.

Conceptual vtable representation:

Bill vtable	Function
Entry 1	Bill::total() (pure virtual)
Entry 2	Bill::type() (pure virtual)
Entry 3	Bill::~Bill()

OutPatientBill vtable	Function
Entry 1	OutPatientBill::total()
Entry 2	OutPatientBill::type()
Entry 3	OutPatientBill::~OutPatientBill()

InPatientBill vtable	Function
Entry 1	InPatientBill::total()
Entry 2	InPatientBill::type()
Entry 3	InPatientBill::~InPatientBill()

How b->total() works:

1. The base pointer stores the address of the derived object.
2. The object has access to its class's virtual-function table.
3. The program identifies the correct overridden function at runtime.
4. The corresponding derived-class function is executed.

Note: This is a conceptual representation. The exact vtable layout is compiler-dependent.

A2.3 Virtual destructor

A virtual destructor is required because we may delete a derived object using a base-class pointer.

```
Bill* b = new OutPatientBill("Ravi", 101, 500, 1200);
delete b;
```

If the base destructor is not virtual, deleting through the base pointer causes undefined behaviour. A virtual destructor ensures that the derived destructor runs first, followed by the base destructor.

Predict the output

Expected output:

```
500 ~OPBill ~Bill
```

Explanation:

- b->show() calls a normal, non-virtual function.

- Inside show(), the call to total() is virtual.
- Therefore, OPBill::total() executes and prints 500.
- When delete b is executed, the derived destructor runs first, followed by the base destructor.

If virtual is removed from ~Bill(), deleting the derived object through a base pointer causes undefined behaviour. The derived destructor is not guaranteed to run, and memory used by derived members may leak. AddressSanitizer may report an error, but the exact report depends on the compiler and runtime.

Alternative concept – dynamic_cast

If total() were not virtual, the program could identify the actual type using dynamic_cast.

```
if (auto* op = dynamic_cast<OPBill*>(b))
{
    // Process outpatient bill
}
else if (auto* ip = dynamic_cast<InPatientBill*>(b))
{
    // Process inpatient bill
}
```

- A virtual function requires one virtual call per bill.
- Using dynamic_cast may require checking up to k bill types.
- Both approaches are generally O(1) per operation for a fixed number of types, but repeated casts increase the work.
- A polymorphic object commonly has a vtable pointer, which uses additional memory.

TASK A3 – DESIGN WITH ABSTRACT CLASSES

A3.1 Payer interface

An abstract class Payer defines a common function called amountCovered().

```
class Payer
{
public:
    virtual double amountCovered(const Bill& b) const = 0;
    virtual ~Payer() = default;
};
```

SelfPay:

```
class SelfPay : public Payer
{
public:
    double amountCovered(const Bill& b) const override
    {
        return b.total();
    }
};
```

Insurance:

```
class Insurance : public Payer
{
    double coverLimit;
    double coPayment;

public:
    Insurance(double limit, double percent)
        : coverLimit(limit), coPayment(percent) {}

    double amountCovered(const Bill& b) const override
    {
        double amount = b.total() * (1 - coPayment / 100);

        if (amount > coverLimit)
            return coverLimit;

        return amount;
    }
};
```

GovernmentScheme:

```
class GovernmentScheme : public Payer
{
    double packageRate;

public:
    GovernmentScheme(double rate) : packageRate(rate) {}

    double amountCovered(const Bill& b) const override
    {
        return packageRate;
    }
};
```

A3.2 Extension – Corporate tie-up

A new payer can be added by creating another derived class.

```
class CorporatePayer : public Payer
{
    double discount;

public:
    CorporatePayer(double d) : discount(d) {}

    double amountCovered(const Bill& b) const override
    {
        return b.total() * (1 - discount / 100);
    }
};
```

The existing payer classes do not need to be modified.

Alternative concept – Function pointers or std::function

A payer policy could instead be stored as a function.

```
#include <functional>

function<double(double)> policy =
    [](double amount)
    {
        return amount * 0.90;
    };
```

- Both approaches provide O(1) work for a simple policy call.
- An abstract Payer class stores its own data, such as cover limit and co-payment.
- A function or std::function stores callable behaviour and may capture the data it needs.
- Inheritance is useful when payer types have different data and behaviour.
- std::function is useful when policies are supplied as interchangeable functions.

TASK A4 – DESIGN THE EXCEPTION STRATEGY

A4.1 Identify the errors

Error	Reason	Exception
Negative charges	Invalid bill amount	BillingError
Claim above cover	Claim exceeds insurance limit	ClaimError
Unknown package	Package code not found	PackageError
Missing patient record	Patient details unavailable	PatientError
Memory allocation failure	Insufficient memory	std::bad_alloc

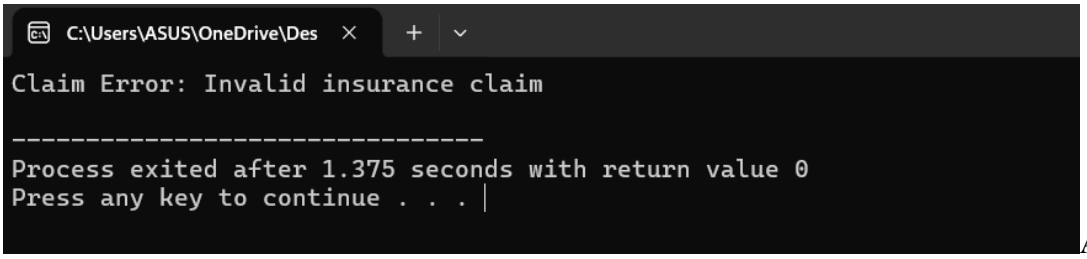
Exception class hierarchy:

```
            std::exception
                  |
          std::runtime_error
                  |
              BillingError
             /     |      \
      ClaimError PackageError PatientError
```

Example:

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;
class BillingError : public runtime_error
{
public:
    BillingError(string message)
        : runtime_error(message)
    {
    }
};
class ClaimError : public BillingError
{
public:
    ClaimError(string message)
        : BillingError(message)
    {
    }
};
class PackageError : public BillingError
{
public:
    PackageError(string message)
        : BillingError(message)
    {
    }
};
class PatientError : public BillingError
{
public:
    PatientError(string message)
        : BillingError(message)
    {
    }
};
int main()
{
    try
    {
        throw ClaimError("Invalid insurance claim");
    }
    catch (const ClaimError& e)
    {
        cout << "Claim Error: " << e.what() << endl;
    }
    catch (const BillingError& e)
    {
        cout << "Billing Error: " << e.what() << endl;
```

Output



A

4.2 Specify exceptions

Functions that may throw:

- calculateBill() – may throw an exception for negative charges.
- approveClaim() – may throw if the claim exceeds the insurance limit.
- findPackage() – may throw if the package does not exist.
- findPatient() – may throw if the patient record is missing.
- Dynamic memory allocation – may throw std::bad_alloc.

Functions that should not throw:

- Destructors
- Simple cleanup functions
- Functions that only release resources

Example:

```
~Bill() noexcept = default;
```

Destructors should not throw because they are often called during stack unwinding. If another exception is already active and a destructor throws, the program may terminate.

Alternative concept – Return codes

Instead of exceptions, functions can return status values.

```
enum Status
{
    SUCCESS,
    NEGATIVE_AMOUNT,
    CLAIM_EXCEEDED
};
```

- Normal execution checks the returned status.
- Error handling requires checking the status at each calling point.
- Return codes do not require exception stack unwinding.
- The caller must pass error information back through return values or output parameters.
- Both methods can take O(1) time for simple validation, but exceptions are useful when errors need to travel through several function levels.

TASK A5 – DESIGN RECOVERY AND RETHROWING

A5.1 Where to catch

The exception-handling process should be:

1. The validation function detects an error and throws an exception.
2. The billing function catches the exception if it needs to log the error.
3. It uses throw; to rethrow the same exception.
4. The main function catches the exception and displays a clear message to the user.

Example:

```
try
{
    processBill();
}
catch (const BillingError& e)
{
    cout << "Billing failed: " << e.what();
}
```

A5.2 Stack unwinding and RAII

Stack unwinding is the process of destroying local objects when an exception moves out of a function.

RAII means Resource Acquisition Is Initialization.

Resources such as memory and files are managed by objects. When an exception occurs, their destructors are automatically called.

Example:

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main()
{
    int* p = NULL;

    try
    {
        p = new int(100);

        throw runtime_error("Error occurred");
    }
    catch (const exception& e)
    {
        cout << e.what() << endl;
    }

    delete p;

    return 0;
}
```

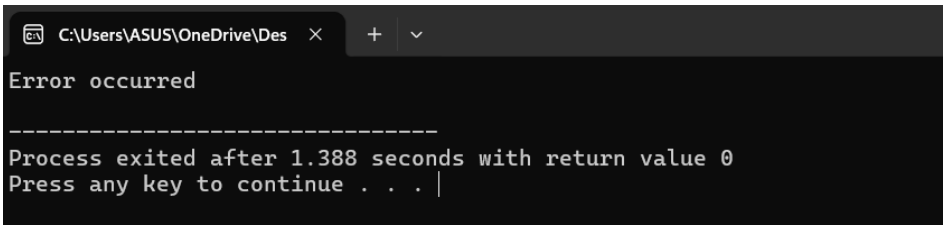
The unique_ptr automatically releases the allocated memory when the exception is thrown.

Predict the output

Expected output:

```
~f ~main caught 42
```

Output



Explanation:

1. main creates the Guard object named main.
2. Function f() creates the Guard object named f.
3. f() throws the integer 42.
4. The local object inside f() is destroyed first, printing ~f.
5. The object in main is destroyed during stack unwinding, printing ~main.
6. The matching catch(int x) handler catches the exception and prints caught 42.
7. "not printed" does not appear because execution leaves the try block immediately when the exception is thrown.

Alternative concept – Manual delete in catch blocks

If memory were released using delete in every catch block:

- Every possible error path must contain the correct cleanup code.
- If a new error path is added, cleanup code must also be added there.
- Missing even one cleanup statement can cause a memory leak.
- RAII performs cleanup automatically.
- Both methods have similar basic cleanup complexity, but RAII is safer and easier to maintain.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND VALIDATE THE POLYMORPHIC BILLING

B1.1 Complete the code – TODO 1, 2 and 3

➢ TODO 1:

```
virtual double total() const = 0;

virtual std::string type() const
{
    return "Bill";
}
```

➢ TODO 2:

```
double total() const override
{
    return consultation + tests;
}

std::string type() const override
{
    return "OP";
}
```

- TODO 3:

```
double total() const override
{
    return days * perDay + surgery;
}

std::string type() const override
{
    return "IP";
}
```

Complete program:

```
#include <iostream>
#include <string>
using namespace std;

class Bill
{
protected:
    string patient;
public:
    Bill(const string& p)
    {
        patient = p;
    }
    virtual ~Bill() {}
    virtual double total() const = 0;
    virtual string type() const
    {
        return "Bill";
    }
    void print() const
    {
        cout << type() << " " << patient
             << ": " << total() << endl;
    }
};
class OutPatient : public Bill
{
private:
    double consultation, tests;
public:
    OutPatient(const string& p, double c, double t)
        : Bill(p)
    {
        consultation = c;
        tests = t;
    }
    double total() const
    {
        return consultation + tests;
    }
    string type() const
    {
```

```
        return "OP";
    }
};
class InPatient : public Bill
{
private:
    int days;
    double perDay, surgery;
public:
    InPatient(const string& p, int d, double rate, double s)
        : Bill(p)
    {
        days = d;
        perDay = rate;
        surgery = s;
    }
    double total() const
    {
        return days * perDay + surgery;
    }
    string type() const
    {
        return "IP";
    }
};
int main()
{
    Bill* bills[2];
    bills[0] = new OutPatient("Ravi", 500, 1200);
    bills[1] = new InPatient("Meena", 3, 2500, 40000);
    double sum = 0;
    for (int i = 0; i < 2; i++)
    {
        bills[i]->print();
        sum = sum + bills[i]->total();
    }
    cout << "Total " << sum << endl;
    for (int i = 0; i < 2; i++)
    {
        delete bills[i];
    }
    return 0;
```

Expected output:

OP Ravi: 1700

IP Meena: 47500

Total 49200

B1.2 Compare with your design

The implementation follows the Part A design:

- Bill is an abstract base class.
- OutPatient and InPatient are derived classes.
- total() is a pure virtual function.
- Each derived class overrides total() and type().
- A virtual destructor ensures correct destruction.

The implementation uses a simple array of base-class pointers to demonstrate runtime polymorphism.

B1.3 Prove runtime binding

If virtual is removed from total(), the program will not compile because:

- The base-class total() is no longer pure virtual.
- The derived-class function is not an override of a virtual function.

- The base-class function is still called through a Bill*, so it cannot provide the intended derived bill calculation.

B1.4 Prove the virtual destructor

If the virtual destructor is removed, deleting derived objects through Bill* causes undefined behaviour. The derived destructors are not guaranteed to run.

Compile using:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address -g taskB1.cpp -o taskB1
```

Run the program and record the actual sanitizer result.

Alternative concept – Templates

Templates can process each bill type separately.

```
template <typename T>
double calculateTotal(const vector<T>& bills)
{
    double sum = 0;

    for (const auto& b : bills)
        sum += b.total();

    return sum;
}
```

- Template processing does not require a virtual call.
- The time per bill is O(1), and processing n bills takes O(n).
- Separate containers may be needed for different bill types.
- It becomes more difficult to keep all bill types in one common list.
- Runtime polymorphism is more suitable when different bill objects must be stored and processed together.

TASK B2 – ENGINEER ERROR CONDITIONS

B2.1 Error tests

The program should test the following conditions:

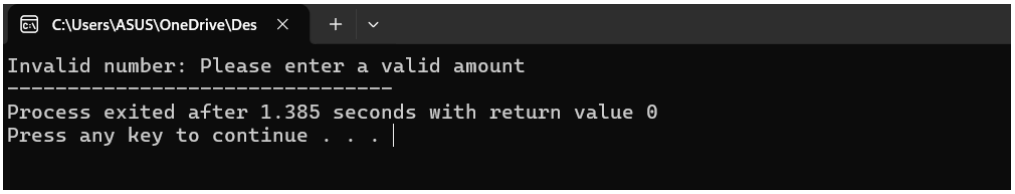
Test	Input	Expected behaviour
Negative charge	-500	Reject with invalid argument
Claim above cover	Bill 60000, cover 50000	Reject with ClaimError
Unknown package	Invalid package code	Throw PackageError
Text instead of number	"abc"	std::invalid_argument
Memory allocation failure	Impossibly large allocation	std::bad_alloc

Example of converting text into a number:

```cpp
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

int main()
{
    string input = "abc";
    double amount;
    stringstream ss(input);
    if (ss >> amount)
    {
        cout << "Amount: " << amount;
    }
    else
    {
        cout << "Invalid number: Please enter a valid amount";
    }
    return 0;
}
```

Expected output:



For memory allocation failure, use a suitably controlled test with an impossibly large allocation request and catch std::bad_alloc. Do not assume every huge request will fail immediately on every operating system.

Predict the output

Given the program in the assignment:

Expected output:

```
ok done caught negative ok done [log too large] caught too large
```

Explanation:

The program processes three amounts:

1. 100.0: Validation succeeds, prints ok, and main prints done.
2. -5.0: Validation throws invalid_argument("negative"). It is not caught by the out_of_range handler inside process(), so it reaches main and prints caught negative.
3. 2e6: Validation throws out_of_range("too large"). The process() function catches it, prints [log too large], and rethrows it. Main catches it and prints caught too large.

Only the third exception is logged inside process() because the local handler catches only std::out_of_range.

Complete the code – TODO 1 and TODO 2

➢ TODO 1:

```
if (billed < 0 || cover < 0)
    throw std::invalid_argument("negative amount");

if (billed > cover)
    throw ClaimError("exceeds cover");

return billed;
```

- TODO 2:

```
std::cout << "audit: " << e.what() << '\n';
throw;
```

Complete program:

```
#include <iostream>
#include <stdexcept>
using namespace std;
class ClaimError : public runtime_error
{
public:
    using runtime_error::runtime_error;
};
double approve(double billed, double cover)
{
    if (billed < 0 || cover < 0)
        throw invalid_argument("negative amount");
    if (billed > cover)
        throw ClaimError("exceeds cover");
    return billed;
}
double settle(double billed, double cover)
{
    try
    {
        return approve(billed, cover);
    }
    catch (const ClaimError& e)
    {
        cout << "audit: " << e.what() << '\n';
        throw;
    }
}
int main()
{
    double cases[][2] = {
        {40000, 50000},
        {60000, 50000},
        {-1, 50000}
    };
    for (auto& c : cases)
    {
        try
        {
            double paid = settle(c[0], c[1]);
            cout << "approved " << paid << '\n';
        }
        catch (const ClaimError& e)
        {
            cout << "claim rejected: " << e.what() << '\n';
        }
        catch (const exception& e)
        {
            cout << "error: " << e.what() << '\n';
```

Expected output:

```
C:\Users\ASUS\OneDrive\Des
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount

--------------------------------
Process exited after 1.327 seconds with return value 0
Press any key to continue . . .
```

Replace throw; with throw e;

- throw; rethrows the same currently handled exception, preserving its original type.
- throw e; throws the caught object as an expression. Because e is a reference to the base type ClaimError here, this particular example still throws a ClaimError object.
- In general, throwing a caught base-class reference by value can cause object slicing if the actual exception is a derived exception type.
- Therefore, throw; is the preferred way to rethrow the same exception.

Alternative concept – std::optional<double>

Instead of throwing exceptions, the function can return an optional value.

```
#include <optional>
using namespace std;

optional<double> approve(double billed, double cover)
{
    if (billed < 0 || cover < 0 || billed > cover)
        return nullopt;

    return billed;
}
```

The caller checks whether a value exists.

Comparison for 1,000,000 claims:

Invalid claims	Return-code / optional approach	Exception approach
0%	Normal validation for all claims	Normal validation for all claims
1%	Check result for each claim	Exception handling for invalid claims
50%	Check result for each claim	Many exceptions and stack unwinding

- Both approaches perform O(n) work for n claims.
- Return codes and optional values are usually suitable for frequent expected failures.
- Exceptions are useful for unusual errors that interrupt normal processing.
- The actual execution time must be measured on your computer.
- std::optional<double> stores a value when successful and a state indicating whether a value exists. Its exact size is implementation-dependent.

TASK B3 – VIRTUAL FUNCTIONS VS SWITCH VS EXCEPTIONS: THE COST

B3.1 Correctness on small data

Create 10 bills with the same values in both implementations.

For example:

Bill	Type	Amount
1	Outpatient	1700
2	Inpatient	47500
3	Outpatient	2500
4	Inpatient	30000
5	Outpatient	1800
6	Inpatient	40000
7	Outpatient	3200
8	Inpatient	25000
9	Outpatient	1500
10	Inpatient	35000

Expected total:

$$1700 + 47500 + 2500 + 30000 + 1800 + 40000 + 3200 + 25000 + 1500 + 35000 = 188200$$

Both the virtual-function implementation and switch implementation must produce:

Total: 188200

B3.2 Increase the data size

Measure both implementations for:

- 1,000 bills
- 1,000,000 bills
- 10,000,000 bills

Compile using optimization:

```
g++ -std=c++17 -O2 -Wall -Wextra taskB3.cpp -o taskB3
```

Use chrono to measure execution time:

```
#include <chrono>
using namespace std::chrono;

auto start = high_resolution_clock::now();

// Execute bill processing here

auto stop = high_resolution_clock::now();

auto duration =
    duration_cast<microseconds>(stop - start);

cout << "Time: " << duration.count()
     << " microseconds\n";
```

Result table to fill after execution:

Number of bills	Virtual function time	Switch time
1,000	Record actual time	Record actual time
1,000,000	Record actual time	Record actual time
10,000,000	Record actual time	Record actual time

B3.3 Cost of exceptions

Measure validation of 1,000,000 bills using:

- 0% invalid bills
- 1% invalid bills
- 50% invalid bills

Compare return-code validation with exception-based validation.

Result table:

Invalid percentage	Return-code time	Exception time
0%	Record actual time	Record actual time
1%	Record actual time	Record actual time
50%	Record actual time	Record actual time

B3.4 Engineering trade-off

Virtual functions:

- Suitable when many different bill types share one interface.
- New bill types can be added without changing existing processing code.
- Runtime dispatch has a small overhead.

Switch statement:

- Suitable when the number of bill types is small and fixed.
- Easy to understand for simple programs.
- Adding new types requires modifying the switch statement.

Exceptions:

- Suitable for unexpected or exceptional errors.
- Avoid using exceptions for normal control flow.
- Frequent exceptions can be expensive.

Return codes:

- Suitable for expected and frequent validation failures.
- Usually efficient for large batches.
- The caller must check every returned status.

Alternative concept – std::variant and std::visit

std::variant can store one of several bill types in a single container.

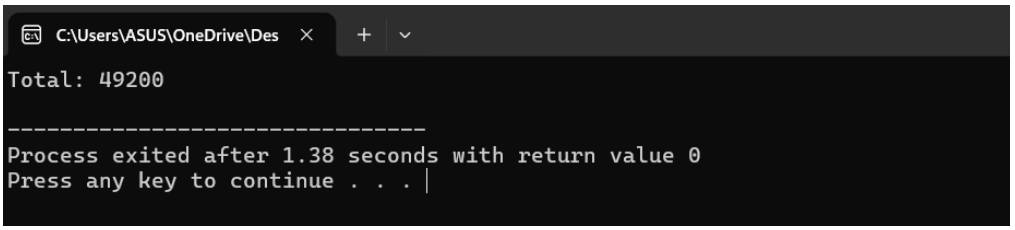
```cpp
#include <iostream>
#include <variant>
#include <vector>
using namespace std;

struct OutPatientBill
{
    double consultation, tests;
    double total() const
    {
        return consultation + tests;
    }
};
struct InPatientBill
{
    int days;
    double perDay, surgery;

    double total() const
    {
        return days * perDay + surgery;
    }
};

int main()
{
    using Bill = variant<OutPatientBill, InPatientBill>;
    vector<Bill> bills;
    bills.push_back(OutPatientBill{500, 1200});
    bills.push_back(InPatientBill{3, 2500, 40000});
    double sum = 0;
    for (const auto& b : bills)
    {
        sum += visit([](const auto& x)
        {
            return x.total();
        }, b);
    }
    cout << "Total: " << sum << endl;
    return 0;
}
```

Output:



Comparison:

Feature	Virtual functions	Switch	Variant
Function selection	Runtime virtual dispatch	Switch condition	std::visit
Storage	Base pointers and separate objects	One structure	Largest alternative plus tag
Adding a type	Add a derived class	Modify switch	Modify variant type and visitors
Common container	Yes	Yes	Yes
Time per bill	O(1)	O(1)	O(1) for a fixed number of alternatives

Run the virtual, switch and variant versions for 10,000,000 bills using the same data and compiler options. Record actual times rather than assuming one method is always faster.

TASK B4 – FINAL ENGINEERING DECISION

B4.1 Implement a practical extension

Add one new bill type and one new payer without modifying existing classes.

- **New bill type – PharmacyBill**

```
class PharmacyBill : public Bill
{
    double medicineCost, serviceCharge;

public:
    PharmacyBill(string p, int b, double m, double s)
        : Bill(p, b), medicineCost(m), serviceCharge(s) {}

    double total() const override
    {
        return medicineCost + serviceCharge;
    }

    string type() const override
    {
        return "Pharmacy";
    }
};
```

- **New payer – CorporatePayer**

```
class CorporatePayer : public Payer
{
    double discount;

public:
    CorporatePayer(double d) : discount(d) {}

    double amountCovered(const Bill& b) const override
    {
        return b.total() * (1 - discount / 100);
    }
};
```

- Example usage:

```
PharmacyBill medicine("Ravi", 105, 2000, 200);

CorporatePayer corporate(10);

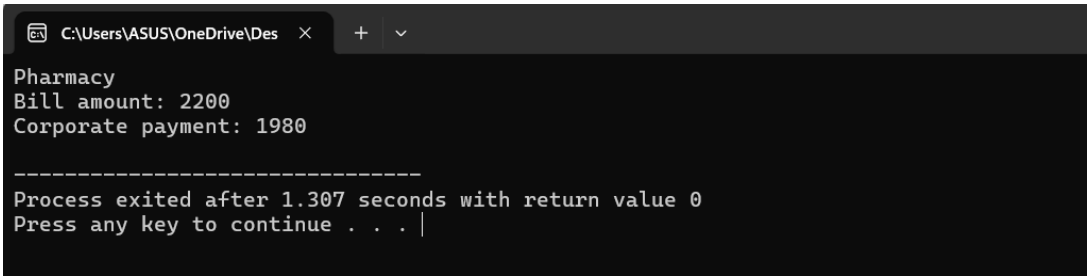
cout << medicine.type() << endl;
cout << "Bill amount: " << medicine.total() << endl;
cout << "Corporate payment: "
     << corporate.amountCovered(medicine) << endl;
```

Full code

```cpp
#include <iostream>
#include <string>
using namespace std;
// Base class Bill
class Bill
{
protected:
    string patient;
    int billNo;
public:
    Bill(string p, int b)
    {
        patient = p;
        billNo = b;
    }
    virtual double total() const = 0;
    virtual string type() const = 0;
    virtual ~Bill() {}
};
// PharmacyBill class
class PharmacyBill : public Bill
{
private:
    double medicineCost, serviceCharge;
public:
    PharmacyBill(string p, int b, double m, double s)
        : Bill(p, b)
    {
        medicineCost = m;
        serviceCharge = s;
    }
    double total() const
    {
        return medicineCost + serviceCharge;
    }
    string type() const
    {
        return "Pharmacy";
    }
};
```

```cpp
// Base class Payer
class Payer
{
public:
    virtual double amountCovered(const Bill& b) const = 0;
    virtual ~Payer() {}
};
// CorporatePayer class
class CorporatePayer : public Payer
{
private:
    double discount;
public:
    CorporatePayer(double d)
    {
        discount = d;
    }
    double amountCovered(const Bill& b) const
    {
        return b.total() * (1 - discount / 100);
    }
};
// Main function
int main()
{
    PharmacyBill medicine("Ravi", 105, 2000, 200);
    CorporatePayer corporate(10);
    cout << medicine.type() << endl;
    cout << "Bill amount: "
         << medicine.total() << endl;
    cout << "Corporate payment: "
         << corporate.amountCovered(medicine) << endl;
    return 0;
}
```

Expected output:



This demonstrates that new bill and payer types can be added without changing existing classes.

B4.2 Check resource safety

Compile the complete program using:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address -g taskB4.cpp -o taskB4
```

Run:

```
./taskB4
```

Test:

- Normal bill calculation.
- Invalid claim.
- Negative bill amount.
- Exception caught by the main function.
- Correct destruction of all dynamically created objects.

Include your actual AddressSanitizer output screenshot in the assignment. A clean run should finish without reported memory errors.

B4.3 Production decision

The polymorphic design with exception handling is suitable for the hospital billing system.

Reasons:

1. It supports different bill types using a common interface.
2. New bill types can be added without modifying existing classes.
3. Different payer policies can be implemented through inheritance.
4. Virtual functions provide runtime polymorphism.
5. Exceptions handle invalid charges and claims clearly.
6. Virtual destructors and RAII help prevent memory leaks.
7. The design is easier to maintain when the hospital group expands to 15 hospitals and processes 200,000 bills daily.

However, performance must be verified through the benchmark results. Frequent validation errors should generally use return codes or optional results rather than exceptions.

Alternative concept – Configuration-based plug-ins

Instead of writing new C++ classes for every bill type and payer, the hospital could store billing rules in configuration files.

For example:

```
Bill Type: OutPatient
Consultation: 500
Tests: 1200

Bill Type: DayCare
Package Rate: 25000
```

The program reads the configuration and calculates bills according to the stored rules.

Comparison:

Feature	C++ inheritance	Configuration-based rules
Adding a new type	Create a derived class	Add a configuration entry
Time per bill	O(1) virtual call	Rule lookup and interpretation
Space	Object data and usually vtable pointer	Stored configuration rules
Flexibility	Requires recompilation and deployment	Rates can often be updated without recompilation
Complexity	More programming work	Requires a reliable rule parser
Risk	Coding errors	Invalid or incorrect configuration

Conclusion: Configuration-based rules are useful for changing prices and simple billing policies. C++ inheritance is more suitable when a new bill type requires new calculation logic or behaviour.